

If not, a `continue` statement causes the execution to move back to the beginning of the loop, which asks the player to enter their move again. Note that we check whether `toTower` is empty; if it is, we return `fromTower, toTower` to emphasize that the move was valid, because you can always put a disk on an empty pole. These first two conditions ensure that by the time the third condition is checked, `towers[toTower]` and `towers[fromTower]` won't be empty or cause an `IndexError`. We've ordered these conditions in such a way to prevent `IndexError` or additional checking.

It's important that your programs handle any invalid input from the user or potential error cases. Users might not know what to enter, or they might make typos. Similarly, files could unexpectedly go missing, or databases could crash. Your programs need to be resilient to the exceptional cases; otherwise, they'll crash unexpectedly or cause subtle bugs later on.

If none of the previous conditions are `True`, `getPlayerMove()` returns `fromTower, toTower`:

```
    else:
        # This is a valid move, so return the selected towers:
        return fromTower, toTower
```

In Python, `return` statements always return a single value. Although this `return` statement looks like it returns two values, Python actually returns a single tuple of two values, which is equivalent to `return (fromTower, toTower)`. Python programmers often omit the parentheses in this context. The parentheses don't define a tuple as much as the commas do.

Notice that the program calls the `getPlayerMove()` function only once from the `main()` function. The function doesn't save us from duplicate code, which is the most common purpose for using one. There's no reason we couldn't put all the code in `getPlayerMove()` in the `main()` function. But we can also use functions as a way to organize code into separate units, which is how we're using `getPlayerMove()`. Doing so prevents the `main()` function from becoming too long and unwieldy.

The `displayTowers()` function displays the disks on towers A, B, and C in the `towers` argument: